

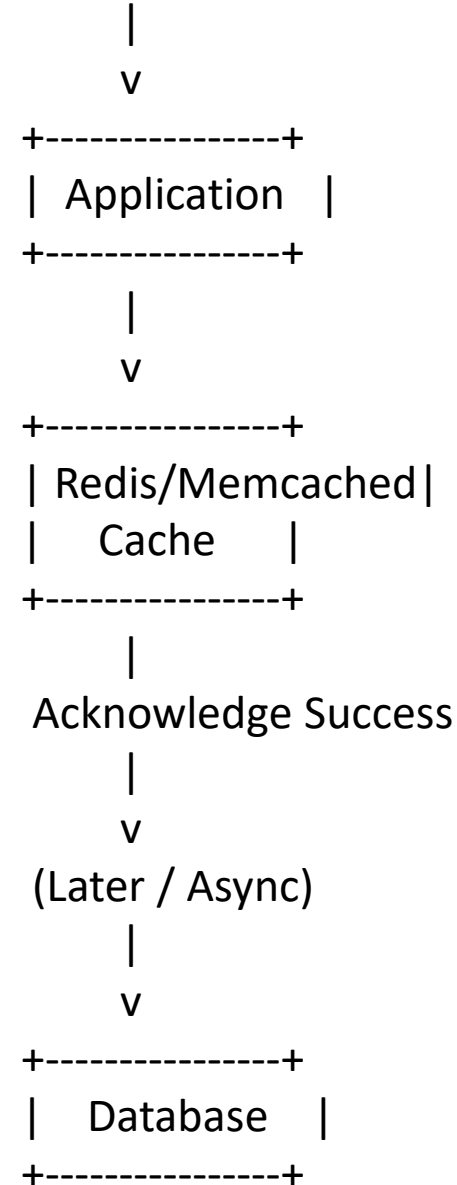
Caching Strategies Part 4

Prepared By: Dr. D. P. Singh

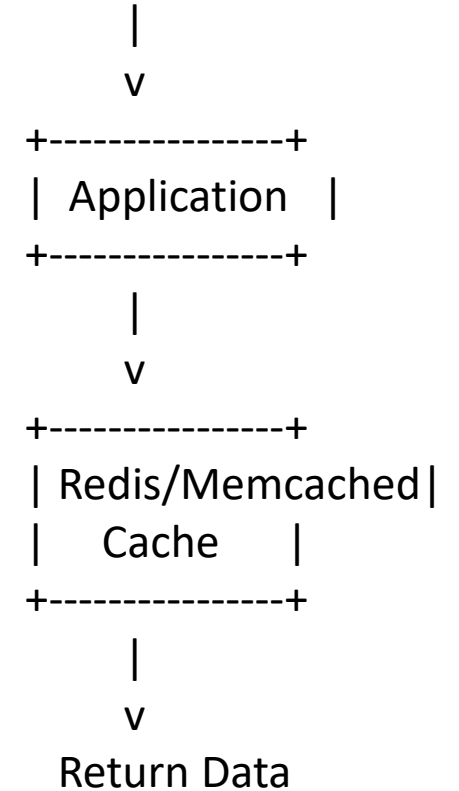
Write-Back (Write-Behind) Overview

- **Write-Back** (also called **Write-Behind**) is a caching strategy where data is written **only to the cache first**.
- The cache acknowledges success immediately, and the database is updated later asynchronously.
- This improves write performance because the application does not wait for the database update.

WRITE REQUEST



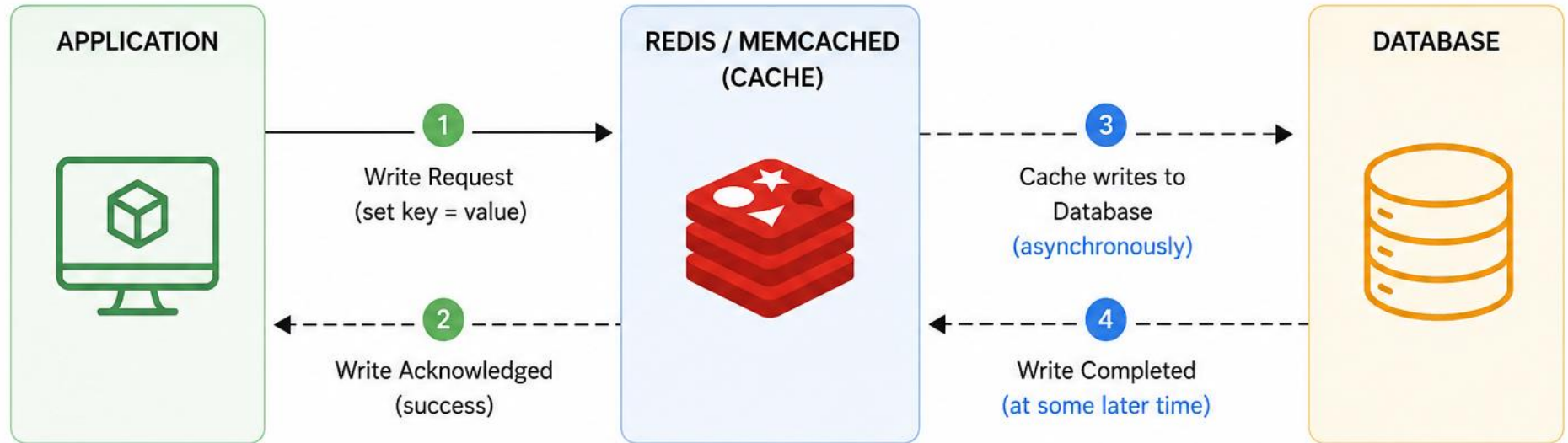
READ REQUEST



Working

Write Back – Working in Redis / Memcached

Write Back means data is written to cache first. The cache then writes (syncs) the data to the database **asynchronously**.



Step 1-2: Fast Response

- Data is written to cache (Redis/Memcached)
- Application gets success response immediately

Step 3-4: Async Sync to DB

- Cache persists the data to the database in background
- Happens asynchronously

Benefits

- Low latency for write operations
- High throughput
- Database is protected from write spikes

Note: If the cache server goes down before syncing, some writes may be lost.
Use Redis persistence (AOF/RDB) or replication for better durability.

Advantages

- ✓ Extremely fast writes
- ✓ Reduced database load
- ✓ Handles write-heavy workloads efficiently
- ✓ Better scalability
- ✓ Multiple updates can be batched before database writes

Disadvantages

- ✗ Risk of data loss if cache crashes before flushing to database
- ✗ Cache and database may be temporarily inconsistent
- ✗ More complex implementation
- ✗ Recovery mechanisms are required

Redis vs Memcached

Redis

- Better choice for Write-Back
- Supports streams and queues
- Can safely buffer writes before database updates

Memcached

- Pure memory cache
- If server crashes, pending writes are lost
- Generally not recommended for critical Write-Back systems

When to Use Write-Back

Best for:

- Analytics systems
- Logging platforms
- IoT data collection
- Metrics aggregation
-
- Gaming leaderboards
- High-volume event processing

Write-Through vs Write-Back

Feature	Write-Through	Write-Back
Database Update	Immediate	Delayed
Write Speed	Slower	Faster
Read Speed	Fast	Fast
Consistency	High	Eventual
Data Loss Risk	Low	Higher
Database Load	Higher	Lower
Complexity	Lower	Higher

Write-Back is chosen when **write performance is more important than immediate consistency**, while Write-Through is chosen when **data consistency is critical**.

- Thank You